IV

Knowledge Representation: AI

KR and Reasoning (KR, KRR) represents information from the real world for a computer to understand and then utilize this knowledge to solve complex real life problems like communicating with human beings in natural language.

There are 5 types of knowledge

- Declerative K

iff kinds of knowledge that can be represented in AI

- objects
- events
- performance
- facts
- meta knowledge
- knowledge Base

5 types of knowledge

Declarative knowledge - It includes objects, concepts, facts & in a declarative sentence.

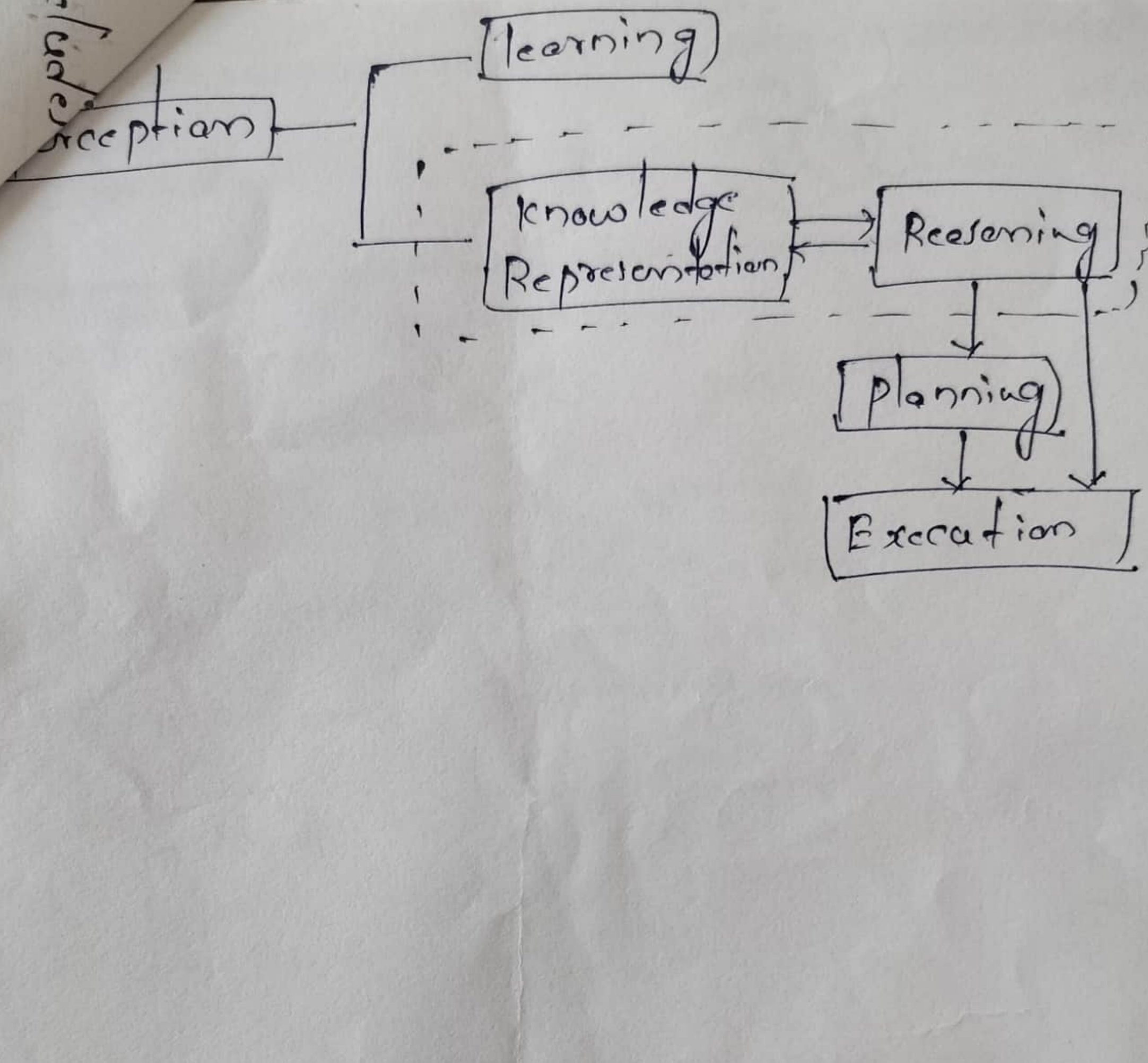
- structural knowledge - It is a basic problem solving knowledge that represents the relation between objects and concepts.

- procedural knowledge - It is responsible for functions, methods, rules etc

- meta knowledge - knowledge about the other types of data.

Heuristic knowledge - expert knowledge in the field or subject.

cycle of KR


learning
Perception
Knowledge Representation
Reasoning
Planning
Execution

9346465760

WARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem
Subject : KRR
H.T. No. :
Date :
Signature of Invigilator

Knowledge Representation is the main thing to solve the problem

- If the KB is wrong the syntax is wrang: and the semantics also wrang.

KB and Reasoning

- logic < P.L / predicate .l
- Rules - If -then
- semantic net - Google Graph
- Frame - slots and fillers (Attributes) (objects)
- script

Train ticket

(means advance to frame).

ontology

ontology is a formal representation of knowledge that defines concepts, relationships and rules with in a particular domain.

- It provides a structured framework for understanding how different entities interact.

concepts (classes) -

It represents categories or types of thing's (e.g., "Disease", "vehicle")

- Relationships (properties) - Define how they are related (e.g: "is part of")
- Individuals (instances) - examples of concepts
- Rules & constraints - Define logical rules (A car must have atleast four wheels")
- ontological engineering is created by
  - knowledge engineers.
  - Domain experts
  - AI researchers and
  - s/w developers

  who will work together to build structured knowledge representation.

important terms related to ontological engineering

1. categories and objects:

   O.E defines structured categories (classes) and their objects (instances) to represent knowledge with in a domain in a hierarchical manner using inheritance.

SWARNA BHARATHI INSTITUTE OF SCIENCE AND TECHNOLOGY
Name :
Year & Sem

Categories

Categories: It defines general concepts in an ontology. They act as blue prints for objects and establish hierarchies and relationships.

objects are specific examples of categories. They inherit properties and relationships from their category.

categories serve to organize and simplify the KB through inheritance.

BSBIT

if we assert fruit is a subclass of Food and Apples are subclass of fruit. then we know that every apple is edible. we say that every apple inherit the property of edibility.

- subclass relations organize categories into a taxonomy or taxonomic hierarchy.

For makes it easy to state facts
categories, either by relating objects
to categories or by quantifying over
their members.

- An object is a member of a category

BBq ∈ BasketBalls

- A category is a subclass of another
category.

Basketballs ⊂ Balls

- All members of a category have
some properties

$x \in$ Basketballs $\Rightarrow$ Round(x)

- member of a category can be recognized
by some properties

Orange(x) ∧ Round(x) ∧ Diameter(x) = 9.5 ∧
$x \in$ Balls
$\Rightarrow x \in$ BasketBalls

- A category as a whole has some properties

Dogs ∈ DomesticatedSpecies.

person ⊆ living Being

relationship (property) Expresions: (Describe how concepts relate to each other)

ex: owns ⊆ person × Car (A person owns a car)

- exhaustive decomposition - If we ensure exhaustive decomposition, every animal must be either Bird or mammal or reptile, but it can't be both.

$x \in Butter \wedge partof(y, x) \Rightarrow y \in Butter$

- we can now say that butter melts at around 30 degrees centigrade.

$x \in Butter \Rightarrow Meltingpoint(x, centigrade(30))$

Events: Events represent real world occurences that happen over time.

- They are a crucial element as they are the starting point for understanding and representing knowledge.

- These events can be individual occurences sequence of actions or even ongoing states.

- Events are fundamental b/c they represent the actions, changes and happenings that an AI system needs to understand and react to.

Events can be represented using techniques including.

1. logic: using predicate logic to represent the facts and relationships associated with an event.
2. Semantic networks: using nodes and edges to represent events and their relationships.
3. Frame based systems: using slots and fillers to represent the characteristics of an event.
4. Knowledge Graphs: using nodes and edges to represent events and their relationships including temporal information.

5. Events:

situations denote the state resulting from executing actions. This approach is called situation calculus.

Situations are logical terms consisting of the initial situation and all situations that are generated by applying an action to a situation.

and member relations are the important ones for categories.

even we went to state relations between categories that are not subclasses of each other.

- A disjoint exhaustive decomposition is known as a partition.

- we say that two or more categories are disjoint if they have no members in common or instances in common. $c_1 \cap c_2 = \phi$

Disjoint({Animals, vegetables})

Exhaustive Decomposition({Americans, Canadians, Mexicans}, {North Americans})

partition({Males, Females}, Animals)

- exhaustive Decomposition of North American is not a partition, b/c some people have dual citizenship. No further decomposition is possible.

Exhaustive. P

ontological Engineering example

In medical diagnosis a team of doctors (domain experts), AI engineers and knowledge engineer collaborate to create an ontology where

Concepts : Diseases, symptoms, Treatments.
Relationships : hassymptom, treatedBy
Instances : covID-19 hassymptom Fever,
treatedBy Antiviral drugs.

Example ontology representation

. category : Disease → Instance : covID-19
. category : symptom → Instance : Fever
. Relationship : covID-19 hassymptom Fever

2. expreniams in ontological engineering

Expreniams in ontological engineering are used to define concepts, relationships, constraints and logical rules within an ontology.

- These expreniams are often represented using formal logic and ontology languages like owL (Web ontology Language) RDF (Resource Description Framework) and Description Logic (DL).

- Important terms related to ontological engineering

- concept (class) expreniam (Defines general categories in an ontology.)

Reasoning with default information 4.2

In the previous Reasoning with categories we have seen default assertions,

- Default can be overriden by more specific information that John has only one leg.

- open and closed worlds.

The following courses will be offered

cs 101, cs 102, cs106, EE 101

course(cs, 101), course(cs, 102), course(cs, 106),

course(EE, 101)

open world and closed world refer to distinct assumptions about the nature of knowledge and reasoning with in a system.

A closed world assumes that what is not known to be true is false,

A open world assumes that what is not known to be true might still be true or atleast unknown.

CWA simplifies reasoning by allowing the system to assume that if a fact is not in its knowledge base, it's not true.

- OWA states that the absence of knowledge about a fact does not imply that fact is false.

It is possible that the fact is true but simply unknown to the system.

ex: ~~This~~ OWA A system trying to identify the capital of a country

If it doesn't have the information about a particular country, it cannot assume the capital is a specific city, it must acknowledge the lack of knowledge.

ex: CWA A DB where all employees are listed and if someone isn't listed, it is assumed they are not employed.

In CWA different names refers to different objects. This is the unique names assumption or UNA:

Course $(d,n) \Leftrightarrow [d,n] = [cs,101] \vee [d,n] = [cs,102] \vee [d,n] = [cs,106] \vee [d,n] = [EE,101]$. ——①

Eq① is called completion.

completion will contain definition.

- The completion is constructed as follows.

1. Gather all the clauses with the same predicate name (p) and the same arity(n).
2. Translate each clause to clark Normal form

CNF means gathering all clauses together by using $\vee$ operator

suppose rules for a atom or atomic sentence

$a \leftarrow b_1$

$\vdots$

$a \leftarrow b_n$

equivalently $a \leftarrow b_1 \vee a \leftarrow b_2 \vee \ldots \vee a \leftarrow b_n$

under the complete knowledge assumption (ckA) if 'a' is true all iff b_i are true

$a \rightarrow b_1 \vee b_2 \vee \ldots \vee b_n$

Thus clauses for mean are

$a \leftrightarrow b_1 \vee \ldots b_n$

$p(t_1 \ldots t_n) \leftarrow$ Body

where $t_1 t_2 \ldots t_n$ are terms.

$p(v_1 \ldots v_n) \leftarrow \exists w_1 \ldots w_m \{v_1 \ldots v_n\} = \{t_1 \ldots t_n\}$

where v_i are newly invented variables & w_i are the variables that appear in the original clause.

- use the same set of v_i for every clause
- This gives us a set of clauses

$p(v_1 \ldots v_n) \leftarrow B_1$

:

$p(v_1 \ldots v_n) \leftarrow B_K$

3. Combine these together into one big disjunctive clause

$p(v_1 \ldots v_n) \leftarrow B_1 \vee \ldots \vee B_K$

4. Form the completion by replacing the $\leftarrow$ with an equivalence.

$p(v_1 \ldots v_n) \Leftrightarrow B_1 \vee \ldots B_K$

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem
Subject : Reasoning
H.T. No. :
Date :
Signature of Invigilator

Horn clauses

Course(CS, 101)

Course(CS, 102)

course(CS, 106)

course(EE, 101)

Course(EE, i) ← Integer(i)

∧ 101 ≤ i ∧ i ≤ 130

Course(CS, m + 100) ← Course(CS, m) ∧ 100 ≤ m

∧ m < 200

clark completion

Course(d, n) ⇔ [d, n] =

{CS, 101}

∨ [d, n] = {CS, 102}

∨ [d, n] = {CS, 106}

∨ [d, n] = {EE, 101}

∨ ∃i [d, n] = {EE, i} ∧

Integer(i) ∧ 101 ≤

i ∧ i ≤ 130

∨ ∃m [d, n] = [CS, m + 100]

∧ Course(CS, m) ∧ 100 ≤ m

∧ m < 200

The clark completion of a set of Horn clauses lists four courses explicitly and also asserts that there is a null clam for every integer from 101 to 130 and that for every CS clam in the 100(undergraduate) series, there is a

class in the 200(graduate) series.

planning: The task of coming up with a sequence of actions that will achieve a goal is called planning.

properties - fully observable, deterministic, finite, static (change happens only when the agent acts) and discrete (in time, action, objects & effects)

These are called classical planning environments.

nonclassical planning is for partially observable or stochastic environments and involves a different set of algorithms and agent designs.

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem

Subject : AI

H.T. No. :
Date :
Signature of Invigilator

- Event calculus is based on points of time rather than on situations.
- A agent should be able to deduce the outcome of a given sequence of actions is called projection task.
- A suitable constructive inference algorithm, agent should be able to find a sequence that achieves a desired effect called planning tasks.
- Discrete events have a definite structure. ex: John trip has begining, middle and end
- categories of events with continuously happening property are called procen categories or liquid event categories.
- Any procen that happens over an interval also happens over any subinterval.

Discrete events have a definite start

- Any process that happens over an interval also happens over any subinterval.
(e ∈ processes) Happens(e, (t1, t4)) (t1 < t2 < t3 < t4)
= Happens(e, (t2, t3))

Here the predicate T represents time.
Here if is subinterval of time lunch hours.

- The liquid event is called temporal substance.
- whereas substance like butter is spatial substance.

- Mental events and objects refers to the internal states and representations with in a system, analogous to human consciousness.
- Mental events are actions or process occuring with in a system. liking thinking or deciding ex: thoughts, feelings, decisions, dreams, realization
- Mental objects are the data or information it manipulates. that an AI system represents and manipulates.

Examples of mental objects are sentences or facts stored in a KB or the sensory information it's processing.

- mental objects are not necessarily physical objects but rather representations with in the system's internal model.

- A mental object is a range of what a person perceives, discovers or learns.

- The term mental object relates to the cognition of human brain.

- Every brain perceives and learns things differently.

- mental events refer to any internal cognitive or conscious process, like thoughts, feelings and decisions.

- mental objects are the entities or concepts that these events relate to.

- understanding and simulating these aspects is crucial for building truly intelligent AI systems, especially in areas like understanding human behavior and creating agents that can reason and

interact with the world.

elaboration of mental events

There are the dynamic internal happenings with in a mind

Thoughts : Deliberations, reasoning processes and mental representation of ideas.

Feelings : emotions, sentiments and subjective experiences.

Decisions : choices made based on mental evaluation and deliberation.

Dreams & Realizations : Internal experiences that can influence behavior and thought.

Mental objects :

These are the entities or concepts that mental events relate to.

physical objects : chair, tree, person..

Abstract concepts : justice, love, freedom

Roles & relationships : like a president, a teacher, a friends.

Reasoning systems for categories:

Reasoning systems in AI categorize information and draw conclusions using various methods.

deductive
Inductive
abductive
analogical
commonsense
monotonic
non-monotonic
Fuzz logic

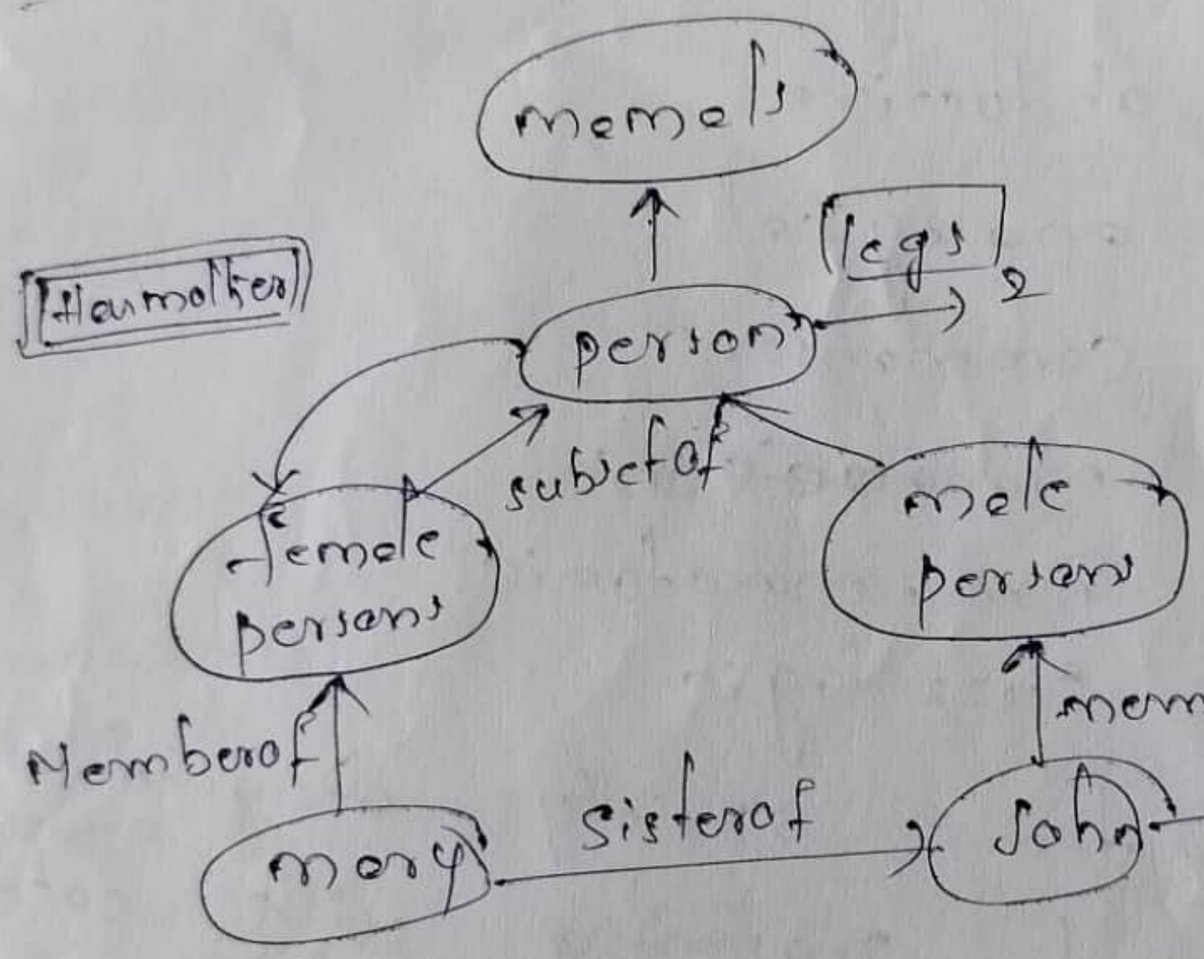
- The systems specially designed for organizing and reasoning with categories.
- There are two closely related families of systems:

Semantic networks provide graphical aids for visualizing a knowledge base and efficient algorithms for inferring properties of an object on the basis of category membership and description

Semantic n/ws :

We use nodes and edges (arcs) to repre

The knowledge
Graphical notation by
using nodes and arcs called existential
graphs, that be called it as
the "logic of the future".



A semantic n/w with four objects
(John, mary, 1 and 2) and four categories,
Relations are denoted by labeled links.

$\forall x \quad x \in persons \Rightarrow [\forall y \; Haumalker(x,y) \Rightarrow y \in female\ persons]$

A person has two legs.

$\forall x \quad x \in persons \Rightarrow legs(x,2)$

Inheritance becomes complicated when an object belongs to more than one category this is called multiple inheritance

- This inheritance will find multiple values / conflicting values for an query.

- Inverse link is another common form of inference.

If P, S Hassister(P, S) ⇔ sisterof(S, P)

Hassister is the inverse of sisterof

- There is also ability to represent default values for categories.

John has one leg if it is fact all persons having two legs.

in a strictly logical KB it is a contradiction.

- In a semantic n/w the assertion that all persons have 2 legs only by specific information one person having one leg.

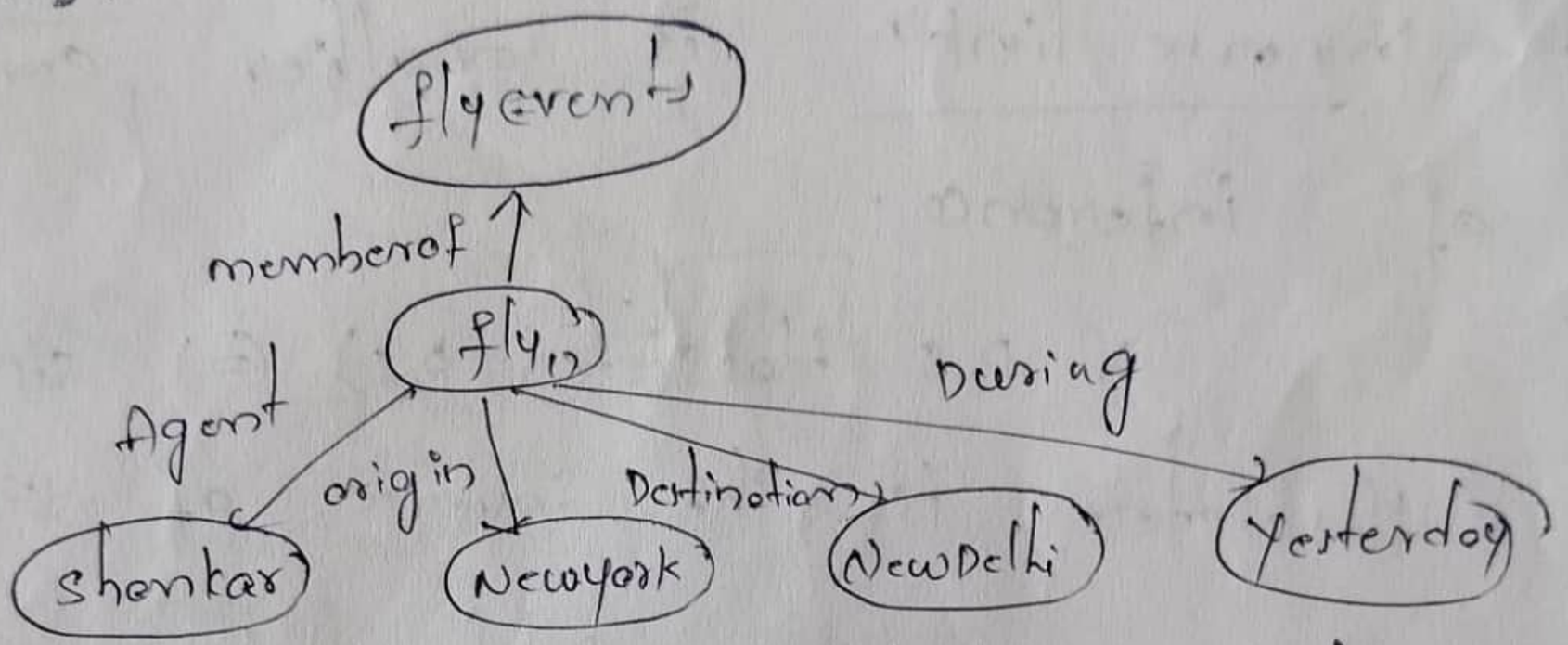
- we can also overridden by the more specific value.

- we can overide by the default number

of legs by category of one legged person, which is a subset of person.
John is a subset of person.

$\forall x \quad x \in persons \wedge x \neq John \Rightarrow legs(x,1)$

A n/w will be updated with more assertions:



A fragment of a semantic n/w showing the representation of the logical assertion.
fly (shankar, Newyork, NewDelhi, yesterday)

SWARNA BHARATHI INSTITUTE OF SCIENCE & TECHNOLOGY [SBIT], KHAMMAM

Name :
Year & Sem : U.G Subject : AI Planning
H.T. No. :
Date :
Signature of Invigilator

- To reach to a particular destination planning is required.
- first we should find the best route and then identify a set of actions to be done at a particular time is called planning.
- planning is nothing but deciding a set of actions to be performed in agent environment based on the perception to achieve the decided goal.
- A planning system requires domain description, action specification and goal description.
- A plan is assumed to be a sequence of actions.
- Each action has its own set of preconditions.
- All the preconditions to be satisfied before performing the action.

.2 types of planning

1. clamical planning environments.
   . fully observable, deterministic, finite,
   static and discrete.
2. Non clamical planning is for partially observable or stachastic environments.
   non clamical planning involves a different set of algorithms and agent design.

- The language of planning problems:

The representation of planning problems are states, actions and goals.

These makes possible for planning algorithms to take advantage of logical structure of the problem.

- The main key to find more efficient and effective algorithm to solve it.

operate if.

Representation of states. planners decompose the world into logical conditions and represent a state as a conjunction of literals.

we will also consider propositional literals.

Ex: poor ∧ unknown might represent one state of a hapless agent.

Folgic literals.

Ex: At(plane, , Melbourne) ∧ At(plane$_2$, sydney)

literals such as At(x,y) or At(Father(Fred), sydney)

are not allowed.

closed world assumption is used any state that are not mentioned in the state assumed false.

Representation of goals: A goal is partially specified state, represented as a conjunction of positive ground literals, such as Rich ∧ famous or At(P_2, Tahiti).

Ex:- for example the state Rich ∧ famous ∧ miserable satisfies the goal Rich ∧ famous.

Representation of actions: An action is specified in terms of the precondition that must hold before it can be executed and the effects that ensue when it is executed.

Ex: An action for flying a plane from one location do another is

Action(Fly(p, From, to),
PRECOND: At(p, from) ∧ plane(p) ∧ Airport(from) ∧ Airport(to)
EFFECT: ¬ At(p, from) ∧ At(p, to))

- This is called an action schema,
- It represents a number of different actions that can be derived by instantiating the variables P, from and to different contents.
- The action name and parameter list Fly(p, from, to) serve to identify the action.
- The precondition is a conjunction of function free positive literals stating what is true in a state before the action can be executed. Any variables in the precondition must also appear in the action's parameter list.

The effect is a conjunction of function free literals, describing how the state changes when the action is executed.

A positive literal P in the effect is asserted to be true in the state resulting from the action, where as negative literal ¬P is asserted to be false.

planning with state-space search:

state space search is a fundamental technique in AI for solving planning problems.

In AI planning the goal is to determine a sequence of actions that transitions from an initial state to a desired goal state

state space search algorithm explore all possible states and actions to find an optimal or feasible solution.

- This approach is crucial in various applications such as robotics, gameplaying, logistics and scheduling.

concepts of state space search

1. state space representation
2. Actions and Transitions
3. Goal state
4. Search tree

state space search Algorithms

1. Breadth first search (BFS)
2. Depth first search (DFS)
3. Iterative Deepening search (IDS)
4. A* search
5. Greedy Best first search
6. Dynamic programming

- SSS (state space search) is most straight forward approach of planning algorithm.

(progression) . Forward state-space search

(Regression) . Backward state-space search

. Heuristics for state-space

- The descriptions of actions in a planning problem specify both preconditions and effects.

It is possible to search in both direction either forward from the initial or backward from the goal.

- we can also use the explicit action and goal representations to derive effective heuristics automatically.

Initial state:

Initial state of the search is the initial state from the planning problem.

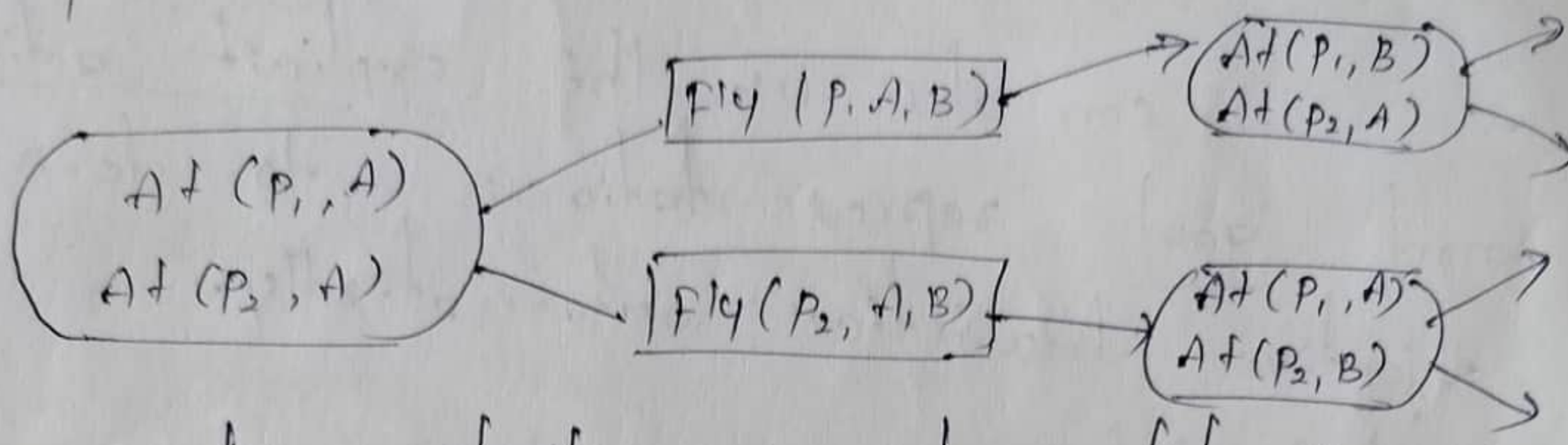
- Each state will be a set of positive ground literals.
- literals not appearing are false.

Actions:

The actions that are applicable to a state are all whose preconditions are satisfied

- The succeson state resulting from an action is generated by adding the positive effect literals and deleting the negative effect literals.
- single sucener function works for all planning problems.

- goal test. checks whether the state satisfies the goal of the planning problem.
- step cost. Different costs for different problems.



Compute whether or not a state is a goal state,

- find the set of all actions that are applicable to a state and
- compute a successor state that is the result of applying an action to a state.
- Algorithm takes as input the statement $P = (O, S_0, g)$ of a planning problem P. (O contains a list of actions).

Backward state space search:

Regression - The goal state must unify with atleast one of the positive literals in the operator's effect.

- It's precondition must hold in the previous situation and these become subgoals. which might be satisfied by the initial conditions.

STRIPS language for planning AI problems:

STRIPS (Standford Research Institute problem solver)
ADL (Action Description language).

In ADL the fly action can be written as

Action (Fly (p: plane, from: Airport, to: Airport),
PRECOND: At(p, from) ∧ (from ≠ to)
EFFECT: ¬ At(p, from) ∧ At(p, to)

STRIPS

- only +ve literals
  POOR ∧ UNKNOWN
- CWA
  unmentioned literals are false
- effect p ∧ ¬q mean add p and delete q
- only ground literals in goals.
  Rich ∧ famous

ADL

- Both +ve & -ve literals
  ¬Rich ∧ ¬famous
- OPWA
  unmentioned literals are unknown.
- effect p ∧ ¬q meand add p and ¬q and delete ¬p and q q.
- quantified variables in goals
  ∃x At(P1, x) ∧ At(P2, x) is the goal of loving P1 and P2 in the same place.

- STRIP is an automated planning technique used to find a goal, by executing a domain from its initial state of domain.
- with STRIPS we can first describe the world (initial state and goal state) by providing objects, actions, preconditions & effects.

- To describe the world we used 2 categories of terms 1. states - Initial state & goals
2. Actionschema - objects, actions, preconditions, effects.

planning Domain Definition language (PDDL)
- A common language for writing STRIPS domain and problem sets.
- In PDDL most of the codes are english words, so that it can be clearly read and understood.
- It is an easy approach to write simple AI planning problems.
- STRIPS also contain states, Actions & goals

States : In STRIPS the states are represented as conjunction of ground, function free and positive literals such as

At(Home) ∧ Have(Banana)

- To describe state CWA is used

Goals : conjunction of literals may contain variables (existential), goal may represent more than one state

eg: At(Home) ∧ ¬ Have(Banana)

. At(x) ∧ Sells(x, Bannanas)

Actions : preconditions that must hold before execution and the effects after execution.

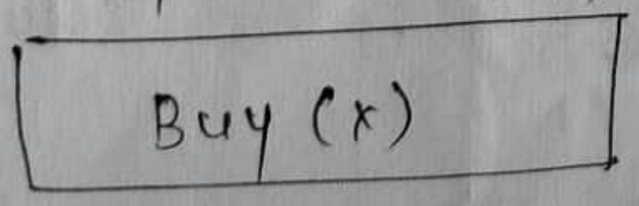
STRIPS Action schema

An action schema includes

. Action name & parameter list (variables)

. precondition : a conjunction of function-free positive literals. The action variables must also appear in precondition.

effect : a conjunction of function free literals (positive or negative)



Add list : +ve literals
Delete list : -ve literals

Example

. Action (Buy(x))
. precondition : At(p), sells(p, x)
. effect Here(x)

Air cargo Transport.

An air cargo transport problem involving loading cargo onto planes and unloading cargo off of planes, flying the planes from one place to another place.

- The problem having three functions / actions.
  . Load, . unload and fly

- The action affect two predicates.
  c - cargo p - plane
  . In (c, p) means that cargo 'c' is inside plane 'p'
  At (x, a) means that an object x (either plane or cargo) is at airport a
  x - object
  a - cargo.

- The following plan is a solution to the problem.

[Load(C_1, P_1, SFO), Fly(P_1, SFO, JFK), Load(C_2, P_2, JFK), Fly(P_2, JFK, SFO)].

C_1, C_2 - cargo

P_1, P_2 - plane

SFO - san_francisco

JFK - ~~John of kanedy~~ John of kanedy

The following plan is a solution to the problem.

load(C_1, P_1, SFO), fly(P_1, SFO, JFK)

load(C_2, P_2, JFK), fly(P_2, JFK, SFO)

A STRIPS problem involving transportation of air cargo b/w airports.

Init(At(C_1, SFO) ∧ At(C_2, JFK) ∧ At(P_1, SFO) ∧ At(P_2, JFK) ∧ cargo(C_1) ∧ cargo(C_2) ∧ plane(P_1) ∧ plane(P_2) ∧ airport(JFK) ∧ airport(SFO))

Goal(At(C_1, JFK) ∧ At(C_2, SFO))

Action(load(c, p, a))

PRECOND: At(c, a) ∧ At(p, a) ∧ cargo(c) ∧ plane(p) ∧ airport(a)

EFFECT: ¬At(c, a) ∧ In(c, p))

c - cargo

p - plane

a - airport

Action(unload(c, p, a),

PRECOND: In(c, p) ∧ At(p, a) ∧ cargo(c) ∧ plane(p) ∧ airport(a)

EFFECT: At(c, a) ∧ ¬In(c, p))

Action (Fly (p, from, to)

PRECOND: At (p, from) ∧ plane (p) ∧ Airport (from) ∧
Airport (to)

EFFECT: ¬ At (p, from) ∧ At (p, to))

planning graphs:

we will explain the planning graph with a simple example planning problem.

Init (Have (cake))

Goal (Have (cake) ∧ Eaten (cake))

Action (Eat (cake)

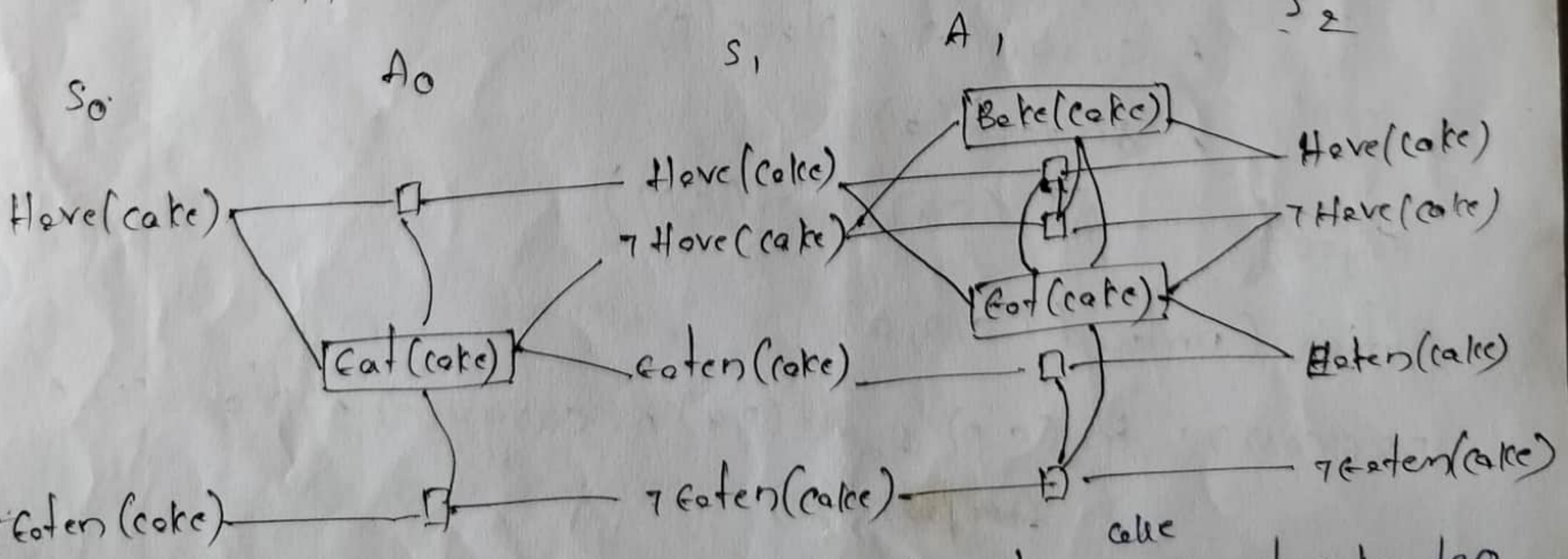
PRECOND: Have (cake)

EFFECT: ¬ Have (cake) ∧ Eaten (cake)

Action (Bake (cake))

PRECOND: ¬ Have (cake)

EFFECT: Have (cake)



A planning graph for the have & eat cake problem up to level S_2.

Rectangles indicate actions (small squares indicate persistence actions)